Pusan National University

Computer Science

PhD Study Report

Alif Wicaksana Ramadhan

A study report

July 7, 2026

Abstract

Write your abstract here. Summarize the research problem, the approach taken, the key findings, and the contribution of this study report in a single self-contained paragraph (typically 150–300 words).

Contents

List of Figures

List of Tables

Part I

Reasoning Model

Chapter 1

Understanding Reasoning

1.1 Reasoning Model

In LLM development, reasoning is defined through a practical engineering lens rather than a philosophical one. Reasoning refers to a model generating intermediate steps before delivering a final response. These steps may be:

- **Visible:** Presented to the user as a step-by-step explanation.
- **Hidden:** Enclosed in special tags (e.g., <think>...</think>) and processed internally.

This process is frequently called "Chain-of-Thought." While it resembles human internal monologue, the underlying mechanism is autoregressive token prediction based on statistical patterns. The goal is to encourage the model to allocate computational resources (tokens) to problem-solving.

Reasoning models excel at tasks where pattern recognition alone fails:

- Logical puzzles and multi-step math problems.
- Complex coding tasks.
- AI agent applications requiring task decomposition, planning, and error recovery.

1.2 Standard LLM Training Pipeline

To understand where reasoning models diverge, one must first understand the training pipeline in the Table 1.1.

Table 1.1: LLM Training Pipeline.

Stage	Process	Objective
Pre-Training	Trained on trillions of tokens of unlabeled text using massive GPU clusters.	Learn "raw language prediction" (next-token prediction) and develop emergent properties.
Post-training: Instruction Tuning	Supervised-finetuning (SFT) on labeled dataset.	Improve the model's ability to follow human instructions for specific tasks (summarization, translation).
Post-training: Preference Tuning	Reinforcement Learning with Human Feedback (RLHF).	Align model outputs with human stylistic choices and subjective preferences.

1.3 Methodologies for Enhancing Reasoning Capabilities

Improving reasoning involves specific techniques typically applied after or on top of the conventional pipeline.

1.3.1 Inference-Time Compute Scaling

This method improves performance at the moment of the user prompt without modifying the model's weights. It trades increased computation for higher accuracy using:

- Chain-of-Thought prompting
- Advanced sampling and voting procedures to identify the best candidate solution.

1.3.2 Reinforcement Learning (RL) for Reasoning

Unlike RLHF, which relies on subjective human judgment, RL for reasoning uses automated or environment-based reward signals.

- **Verification:** Rewards are given for verifiable correctness (e.g., a correct math answer or functional code).
- **Weight Updates:** This process modifies the model's weights through trial and error.

1.3.3 Knowledge Disitllation

This involves transferring reasoning patterns from a large, high-performing model ("teacher") to a smaller, more efficient "student" model. This is typically achieved via SFT using high-quality reasoning datasets generated by the stronger model.

1.4 Pattern Matching vs. Logical Reasoning

A critical distinction exists between how LLMs "reason" and how deterministic rule-based systems operate.

- Rule-Based Systems (Symbolic Logic): These follow strict, hand-crafted heuristics and formal rules. They guarantee logical consistency and correctness if the premises are true.
- Statistical LLMs (Pattern Matching): These identify statistical associations. For example, if a model says the capital of Germany is Berlin, it is recalling a high-probability association, not deducing it.
- Implicit vs. Explicit Reasoning: Conventional models like GPT-4o can simulate reasoning in familiar contexts because they have encountered similar scenarios in their training data.

However, they struggle with novel scenarios or highly intricate multi-step relationships because they do not explicitly track logic or contradictions.

1.5 Report Structure

The shift toward reasoning models is a major industry trend characterized by the following developments:

- **Competitive Landscape:** Major advancements include OpenAI's o1 (September 2024) and DeepSeek's R1 (January 2025). Other major players include Anthropic (Claude 4), Google (Gemini 2.5), and Alibaba (Qwen3).
- **Model Unification:** OpenAI's leadership has stated that GPT-4.5 (Orion) will be their last non-CoT model, with future goals to unify "thinking" models (o-series) and "standard" models (GPT-series).
- **Design Efficiency:** Reasoning is not always desirable. It is "overkill" for simple tasks like translation or basic summarization. Reasoning models are:

Chapter 2

Foundational Text Generation with Pre-trained Large Language Models

2.1 The Reasoning Model Development Framework

The transition from a standard LLM to a reasoning-capable system follows a four-stage mental model. Current focus remains on Stage 1: LLM Basics, which involves the initialization of a conventional LLM and the implementation of basic text generation.

Table 2.1: Basic Text Generation Pipeline.

Stage	Focus Area	Objective
1	LLM Basics	Load pre-trained weights and implement text generation functionality.
2	Measuring Performance	Evaluate response qualities using benchmarks and judgment-based metrics.
3	Reasoning Without Training	Explore inference techniques like self-refinement and advanced voting.
4	Reasoning With Training	Improve capabilities via Reinforcement Learning and Distillation.

Reasoning techniques are typically applied on top of a conventional (base) LLM. A "base" model is one that has undergone pre-training but has not yet been instruction- or preference-tuned. This approach allows developers to isolate which capabilities originate from the reasoning methods themselves versus the underlying model.

2.2 The Tokenization Pipeline

Tokenization is the critical bridge between human-readable text and the numerical data (Token IDs) required by neural networks.

The system utilizes a subword-based method known as Byte Pair Encoding. This allows the model to represent common words as single units and rare words as subword components (e.g., "Explain" may be split into "Ex" and "plain").

The Qwen3Tokenizer features a vocabulary of approximately 151,000 tokens.

- Large Vocabulary Pros: Allows more words to be represented as single tokens, resulting in shorter sequence lengths and lower overall processing time.
- Large Vocabulary Cons: Increases model size and the per-token compute cost for calculating next-token probabilities.

2.3 The Autoregressive Generation Process

Text generation in LLMs is autoregressive, meaning the model generates one token at a time, and each generated token is appended to the input for the next iteration.

2.3.1 Mechanics of a Single Iteration

1. Forward Pass: The model processes the full input sequence (e.g., 6 tokens).
2. Matrix Output: The model returns a matrix (e.g., 6 x 151,936). Each row represents scores for every word in the vocabulary.
3. Last Position Extraction: Only the prediction at the final position is used, as it indicates the next unseen token.
4. Greedy Decoding (Argmax): The system selects the single highest-scoring next token by finding the index of the largest value in the final row of the output matrix.

Chapter 3

Inference-Time Scaling

3.1 The Paradigm of Inference-Time Scaling

In the development of reasoning models, there are two primary strategies to enhance performance: increasing training compute or increasing inference compute.

3.1.1 Core Definition and Mechanics

Inference-time scaling involves allowing a model to do "more work" per question after it has already been trained. Rather than changing the model's weights, this approach utilizes the same trained model to generate higher-quality responses by:

- **Generating more tokens:** Allowing the model to "think" longer.
- **Sampling multiple answers:** Generating a variety of potential solutions to find the most robust one.
- **Successive refinement:** Iteratively reviewing and improving an initial response.

3.1.2 The Scaling Trade-off

There is a direct correlation between the resources spent during inference and the resulting accuracy. As illustrated in the source data, spending more compute on the fly results in a sharper upward curve for benchmark accuracy. The primary disadvantage is that every additional token requires another forward pass through the model, increasing latency, compute cost, and API expenses.

3.2 Primary Methodologies for Improved Reasoning

The following three methods represent the practical paradigms for scaling reasoning performance without retraining the model.

3.2.1 Method 1: Chain-of-Thought (CoT) Prompting

CoT prompting is a sequential technique that modifies the input prompt to encourage the LLM to generate a "reasoning chain" or a step-by-step explanation before arriving at a final answer.

- **Implementation:** Simple instructions such as "Explain step by step" or "Let's think step by step" are appended to the prompt.
- **Mechanism of Improvement:**
  - **Self-Correction:** Walking through steps allows the model more opportunities to correct its own trajectory.
  - **Pattern Alignment:** Most high-quality math and logic datasets contain detailed solutions; CoT aligns the model with these learned patterns.
- **Limitations:** CoT is not universally beneficial. On simple problems, it may lead to "overthinking," where the model generates erroneous explanations that mislead it toward a wrong conclusion.

3.2.2 Method 2: Parallel Sampling and Self-Consistency

This method involves generating multiple independent responses for the same query and using a "voting" mechanism to select the most frequent answer.

- **Implementations:** The model generates N diverse candidate answers. The system then selects the most common final result.
- **Synergy:** This technique is often combined with CoT prompting to generate multiple reasoning chains, significantly increasing the probability of reaching the correct conclusion.
- **Production Use:** This approach is a staple of advanced systems, including high-tier models like Claude 4.

3.2.3 Method 3: Iterative Self-Refinement

A more advanced approach where the model reviews its own reasoning and answers across multiple steps, progressively improving the output.

3.3 Technical Controls for Output Diversity

To enable methods like Self-Consistency, the model must be able to generate diverse responses rather than the same response every time. This requires moving beyond "greedy decoding" (always picking the highest-probability token).

3.3.1 Logits and Probability Distribution

The generation process involves three stages:

1. **Tokenization:** Converting text to token IDs.
2. **Logit Generation:** The model computes raw scores (logits) for every possible next token in the vocabulary (e.g., 151,936 unique tokens).
3. **Token Selection:** Selecting the index with the highest score.

3.3.2 Temperature Scaling

Temperature is a parameter that adjusts the "sharpness" of the logit distribution before it is converted into probabilities via the Softmax function.

Table 3.1: Temperature Settings and Their Effect on Output.

Temperature Setting	Effect on Distribution	Model Behavior
0.0 (Greedy)	Maximum sharpness	Always picks the single most likely token.
Low (< 10)	Sharper distribution	Increases confidence; the gap between the top token and others grows.
1.0	No change	Uses the model's original raw
High (> 1.0)	Flatter distribution	Increases diversity; loweer-ranked tokens become more likely to be selected.

3.3.3 Multinomial Sampling

Instead of always selecting the maximum logit, multinomial sampling draws a token at random based on the weighted probabilities. This randomness is essential for generating the diverse candidate answers required for Self-Consistency.

3.4 Balancing Diversity with Coherence: Top-p Filtering

High temperature settings can occasionally cause the model to sample "weird" or nonsensical tokens (e.g., sampling "mistress" when the query is "The capital of Germany is..."). To prevent this, Top-p Filtering (also known as Nucleus Sampling) is implemented.

3.4.1 The Four-Step Top-p Process

This filter ensures that the model only samples from the most plausible tokens:

1. **Sort:** Arrange token probabilities in descending order.
2. **Cumulative Sum:** Calculate the running total of probabilities.
3. **Apply Threshold:** Select the smallest subset of tokens whose cumulative probability exceeds the threshold $p(e.g., p = 0.9)$.
4. **Renormalize:** Rescale the remaining probabilities so they sum to 1.0, creating a new, valid distribution for sampling.

By dropping low-probability tokens, the model maintains coherence while still allowing for the variability needed for advanced inference scaling strategies.

Chapter 4

Training Reasoning Models with Reinforcement Learning

4.1 Core Training Paradigms: Scaling and Methods

4.1.1 Inference-Time vs. Training-Time Scaling

Reasoning performance and answer accuracy can be improved through two distinct compute investment strategies:

- **Inference-Time Scaling:** Increases accuracy by spending more computation per generated answer (e.g., advanced text generation and voting).
- **Training-Time Scaling:** Improves accuracy by investing additional computation during the training phase to modify model weights.

4.1.2 The Role of Reinforcement Learning (RL)

RL is typically applied as a post-training stage following pre-training and instruction fine-tuning.

- **Pre-training:** Primarily builds the model's knowledge base by training it to predict the next token.
- **Reinforcement Learning:** Shapes how the model uses its knowledge, specifically focusing on its reasoning behavior. It allows for the optimization of whole outputs (e.g., answer correctness) rather than individual tokens.

4.2 Comparison of RL Frameworks: RLHF vs. RLVR

Table 4.1 contrasts the two reinforcement-learning frameworks across their goals, reward signals, and scalability.

Table 4.1: Comparison of RLHF (preference tuning) and RLVR (reasoning training).

Feature	**RLHF (Preference Tuning)**	**RLVR (Reasoning Training)**
Primary Goal	Aligning model outputs with human preferences.	Improving correctness on complex tasks (math, code).
Reward Signal	Human preference labels (subjective).	Verifiable rewards (deterministic/objective).
Reward Model	Requires training an additional LLM as a reward model.	Uses a deterministic verifier (e.g., a math verifier).
Complexity	High; requires human annotators and dual-model training.	Lower; collapses into a single training loop.
Scalability	Limited by human labeling effort and noise.	Scales naturally to large datasets without manual labels.

4.2.1 RLVR Practical Advantages

- **Deterministic and Reproducible:** Avoids the inconsistency inherent in human annotations.
- **Domain Specificity:** Particularly effective for domains with reliable verification signals, such as mathematics and programming.
- **Resource Efficiency:** Removes the cost of maintaining a separate reward model often comparable in size to the base LLM.

4.3 The GRPO Algorithm

Group Relative Policy Optimization (GRPO) is the preferred policy optimization algorithm for implementing RLVR in modern reasoning models.

4.3.1 GRPO vs. PPO

Traditional PPO requires a separate "value model" to estimate a value function, which increases computational overhead. GRPO is more resource-friendly because it derives its learning signal from relative comparisons within a group of sampled responses.

4.3.2 Technical Implementation (The Chef Analogy)

The mechanics of GRPO are illustrated through the analogy of a chef running a delivery service:

1. **Prompt:** A customer request (e.g., a request for lasagna).
2. **Rollouts:** The chef prepares several recipe variations for that single request.
3. **Completions:** The final dishes served to the customer.
4. **Reward:** The customer provides feedback after tasting the entire dish (whole-output evaluation).
5. **Advantages:** The chef compares the dishes relative to each other to identify which variations were superior.
6. **Logprobs:** The chef tracks how characteristic each dish was of their current "cooking style."
7. **Policy Gradient Loss:** The chef tweaks their style to reinforce successful choices.
8. **KL Loss Term:** The chef consults their original cookbook (reference model) to ensure the changes aren't too drastic (style-preservation).

4.3.3 Key Components of the GRPO Pipeline

1. **Sampling Rollouts:** The LLM generates multiple complete answers (rollouts) for a given prompt using temperature scaling and top-p sampling.
2. **Calculating Rewards:** A verifier (e.g., a math script) checks for final answer correctness. In reasoning tasks, binary rewards (1 for correct, 0 for incorrect) are common.
3. **Computing Advantages:** Advantages capture how a rollout performed relative to the group mean.
   - **Formula:**
   $$A_i = \frac{r_i - \mu_r}{\sigma_r + \varepsilon} \tag{4.1}$$
   - **Positive Advantage:** Increases the likelihood of the actions that produced that rollout.
   - **Negative Advantage:** Decreases the likelihood.

4. **Sequence Log-probabilities:** Measures how likely the model considers the generated tokens under its current parameters. Unlike standard LLM training, GRPO often uses sequence-level logprobs because the reward applies to the entire response.

5. **KL Regularization (Optional):** A penalty term that prevents the updated model from drifting too far from the original pre-trained model. While part of the original formulation, some developers omit it to simplify implementation.

Chapter 5

Distilling Reasoning Models for Efficient Artificial Intelligence

Model distillation is a training-time technique where a smaller "student" model is trained to replicate the outputs—specifically the reasoning traces and final answers—of a much larger "teacher" model. This approach addresses the high computational and financial barriers associated with massive reasoning models like the 671-billion-parameter DeepSeek-R1.

Critical Takeaways:

* Efficiency Gains: Distillation training is significantly faster and less resource-intensive than Reinforcement Learning from Verifiable Rewards (RLVR). In documented setups, distillation required only 3 hours and 15 GB of RAM, compared to 12 hours and 70 GB for RLVR. * Superior Performance: Small models trained through distillation often outperform comparable models trained via reinforcement learning alone. * Practicality: "Hard distillation," which uses only the teacher's text outputs rather than complex probability distributions (logits), is the most common and practical method for Large Language Model (LLM) development. * Data Quality: The effectiveness of the student model is fundamentally tied to the quality of the teacher's reasoning traces. High-performing teachers (e.g., DeepSeek-R1 with 91% accuracy) provide the necessary supervision for the student to improve beyond its base capabilities.

5.1 Foundational Concepts of Model Distillation

Model distillation serves as a method to scale down the capabilities of massive systems into formats that can be deployed on local hardware or less expensive infrastructure.

5.1.1 The Teacher-Student Dynamic

* Teacher Model: A large, high-performance LLM (e.g., DeepSeek-R1) that generates "supervision" data. * Student Model: A smaller LLM (e.g., Qwen3 0.6B) that is fine-tuned

to reproduce the teacher's reasoning path and final conclusion.

5.1.2 Comparison of Distillation Types

The documents identify three primary approaches to distillation, summarized in the table below:

Distillation Type Training Target Requirements Practicality for LLMs Hard Distillation Teacher-generated text tokens. Access to text outputs only. High: Standard for LLMs; compatible with proprietary APIs. Soft Distillation Teacher's full probability distribution (logits). Access to teacher logits and identical tokenizers. Low: Computationally expensive; logits are often hidden by providers. Combined Both text tokens and probability distributions. Full model access (Teacher + Student). Medium: Common in computer vision; rarer in LLM distillation.

5.2 Dataset Generation and Preparation

The success of distillation depends on the creation of a high-quality synthetic dataset consisting of complex reasoning tasks.

5.2.1 Synthetic Data Generation

The documented process utilized 12,000 math problems from the Mathematics Aptitude Test of Heuristics (MATH) dataset.

- **Source:** Problems were processed by DeepSeek-R1 to generate reasoning traces.
- **Cost Efficiency:** Using an API (OpenRouter) to generate 12,000 solutions cost approximately $50, which is significantly lower than the cost of training a massive model from scratch.
- **Performance Baseline:** DeepSeek-R1 achieved 90.6% accuracy on the training set and 91.2% on the MATH-500 test set, providing a high-quality target for the student model.

5.2.2 Formatting for Reasoning

To facilitate clear parsing, reasoning traces are typically enclosed in `<think>...</think>` tags. This serves several purposes:

- **User Interface Control:** Allows applications to hide verbose reasoning while showing the final answer.

- **Consistent Formatting:** Helps the model learn a clear boundary between internal "thought" and the final output.
- **Standardization:** Aligns the training with modern reasoning model conventions (e.g., Qwen3 and ChatGPT).

5.3 Preprocessing and Building Training Examples

5.3.1 Tokenization Pipeline

The process follows a three-stage pipeline for each sample:

1. **Prompt Rendering:** The math problem is formatted into a chat prompt (e.g., using `<|im_start|>user` tags).
2. **Answer Formatting:** The teacher's reasoning trace and final answer are combined into a single target string.
3. **Concatenation:** The prompt and answer are joined into a single token sequence ending with an end-of-sequence (`<|im_end|>`) token.

5.3.2 Filtering and Sequence Management

Reasoning traces can be excessively long, which increases computational requirements.

- **Average Length:** The initial dataset averaged 2,946 tokens per response.
- **Outliers:** Some traces reached up to 42,005 tokens.
- **Filtering Strategy:** To keep costs reasonable and fit within hardware constraints, the dataset was filtered to a maximum length of 2,048 tokens. This removed 5,305 of the original 12,000 examples, resulting in a training set of 6,695 high-quality examples.

5.4 Distillation Training Methodology

Distillation is implemented as a supervised fine-tuning task using cross-entropy loss.

5.4.1 Cross-Entropy and Log-Probabilities

Training focuses on Next-Token Prediction. The model is penalized based on how much probability it assigns to the "correct" token (the one generated by the teacher).

- **Log-Probability:** Measures the model's confidence in the correct next token.
- **Cross-Entropy Loss:** The negative average of these token log-probabilities. A value closer to 0 indicates the student is highly confident in the teacher's reasoning path.

5.4.2 Answer-Only Loss

A critical technical detail in distillation is the use of Answer-Only Loss.

- **Mechanism:** The loss is computed only on the tokens generated by the teacher, not the tokens in the prompt.
- **Logic:** The model's task is to generate the answer conditioned on the prompt. Penalizing the model for reproducing the prompt tokens (which are already provided as input) is counterproductive.

5.4.3 The Training Loop

Distillation training iterates over the dataset for multiple epochs.

- **Epoch:** One complete pass through the training data. Shuffling the data each epoch helps the model generalize.
- **Metrics:** Progress is tracked via Training Loss (measured on optimized samples) and Validation Loss (measured on a held-out set). Validation loss is the more reliable metric for assessing whether the student's improvements will generalize to new problems.

Part II

Graph RAG & Ontology

Chapter 1

Understanding Graph RAG and Ontologies

1.1 Retrieval-Augmented Generation Fundamentals

Retrieval-Augmented Generation (RAG) augments a large language model with a non-parametric memory, so that generated text is conditioned not only on the parameters learned during pre-training but also on documents retrieved from an external corpus at inference time [14]. The design was introduced to address two structural weaknesses of purely parametric models: their knowledge is frozen at training time and cannot be updated without re-training, and they offer no straightforward mechanism for attributing a statement to a source [14]. By retrieving supporting passages on demand, RAG was shown to generate more specific and factual language than a comparable parametric-only sequence-to-sequence baseline [14], and it has since become a dominant paradigm for grounding language models on private or fast-changing knowledge [6].

Beyond factual grounding, RAG mitigates hallucination, injects domain-specific knowledge that the base model never observed, and keeps answers current without the cost of continual fine-tuning [6], [18]. These properties explain its rapid adoption across open-domain question answering, enterprise search, and knowledge-intensive assistants [6].

1.1.1 The Retrieve-then-Generate Pipeline

A standard RAG system operates in two stages: a retriever first selects a small set of passages from the corpus that are most relevant to the user query, and a generator then conditions on the query together with those passages to produce the answer [14]. The literature terms this "retrieve-then-generate" arrangement *naive* RAG, distinguishing it from later variants that add query rewriting, re-ranking, and iterative retrieval around the same core loop [6]. The quality of the final answer is therefore bounded by the retriever: if the relevant evidence is never retrieved, the generator cannot recover it [6].

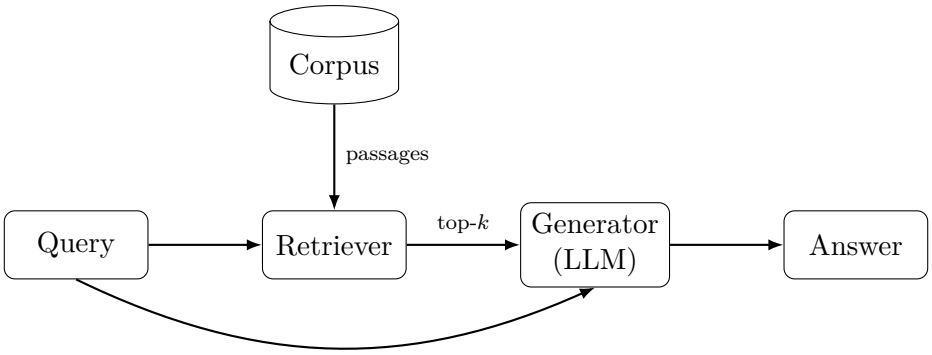

Figure 1.1: The retrieve-then-generate pipeline of naive RAG: a retriever selects the top-k passages from the corpus, and the generator conditions on both the query and those passages to produce the answer [14].

1.1.2 Embeddings and Vector Similarity Search

Most modern retrievers are *dense*: passages and queries are each mapped to a fixed-length vector by a neural encoder, and relevance is estimated by similarity in that embedding space rather than by lexical overlap [12]. Dense Passage Retrieval demonstrated that dual-encoder embeddings trained on question–passage pairs substantially outperform sparse lexical baselines on open-domain question answering [12], and sentence-level encoders such as Sentence-BERT made it practical to embed and compare large collections efficiently [19]. At query time the system performs an approximate nearest-neighbour search over the pre-computed passage vectors to obtain the top-k candidates [12].

1.2 Limitations of Vector-Based Retrieval

Although dense vector retrieval is effective for locating passages that are semantically similar to a query, it treats the corpus as a flat collection of independent chunks and discards the relational structure that connects them [18]. This flat view is the root of several documented failure modes, and it is the primary motivation for introducing graph structure into retrieval [4], [18].

1.2.1 Multi-Hop and Relational Queries

Questions whose answers require chaining several facts together are difficult for naive RAG, because the individual passages that must be combined are often not individually similar to the query and may not co-occur in any single chunk [18]. Multi-hop question answering benchmarks were created precisely to expose this weakness, requiring a system to find and reason over multiple supporting documents rather than retrieving a single relevant passage [26]. Because vector similarity captures topical closeness but not the explicit relations between entities, it provides no reliable way to follow such reasoning

chains [18].

1.2.2 Global vs. Local Questions

A second limitation concerns *global* or thematic questions directed at an entire corpus, such as "what are the main themes in this dataset?". Such questions are inherently query-focused summarization tasks rather than retrieval tasks, and top-k passage retrieval cannot answer them because no small set of chunks contains the global answer [4]. Naive RAG consequently underperforms on comprehensiveness and diversity for this class of question, whereas a graph-structured index that pre-computes summaries of related entities was shown to improve substantially on both axes [4].

1.3 Knowledge Graphs and Ontologies

A knowledge graph represents knowledge as a graph whose nodes are entities of interest and whose edges denote relationships between them, providing an explicit and machine-readable model of a domain [9]. Unlike free text, this structure makes relations first-class objects that can be queried, validated, and reasoned over [9], [10]. Knowledge graphs have been adopted at scale by both industry and academia precisely because they can integrate diverse, dynamic, large-scale collections of data [9].

1.3.1 Entities, Relations, and Triples

The atomic unit of a knowledge graph is the *triple*: a subject entity, a predicate (relation), and an object, written as (subject, predicate, object), which asserts a single labelled edge between two nodes [9]. A collection of such triples forms a directed, edge-labelled graph, and this simple model underlies the surveyed families of graph data models [1], [9]. Representing knowledge as triples is what allows a graph to be traversed to answer relational and multi-hop questions that flat text cannot support directly [10]. Figure 1.2 shows a small example graph assembled from such triples.

1.3.2 Ontology Components: Classes, Properties, and Individuals

Whereas a knowledge graph records instance-level facts, an *ontology* specifies the schema those facts must obey. Formally, an ontology is an explicit specification of a conceptualization—a description of the concepts and relationships that can exist in a domain [7]. Its core building blocks are classes (types of entity), properties (the relations and attributes that may hold), and individuals (the concrete instances), together with axioms that constrain

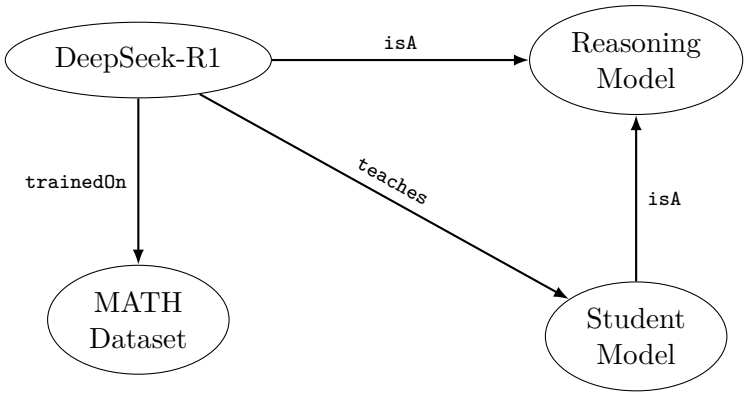

Figure 1.2: A knowledge graph is a set of (subject, predicate, object) triples, each asserting one labelled edge between two entities [9].

how they may be combined [9]. By fixing this vocabulary, an ontology gives the graph a shared, consistent meaning and enables logical inference over it [7], [9].

1.3.3 Standards: RDF, RDFS, and OWL

These ideas are standardized by the World Wide Web Consortium as part of the Semantic Web vision, in which data is published with well-defined meaning so that machines can process it [2]. The Resource Description Framework (RDF) provides the triple-based data model and abstract syntax for representing graph data on the Web [24], while the Web Ontology Language (OWL) supplies the expressive constructs—class hierarchies, property characteristics, and logical axioms—needed to define rich ontologies and to support automated reasoning [25]. Together they form the standard stack on which interoperable knowledge graphs are built [9].

1.4 Graph RAG versus Vector RAG

Graph RAG is the family of RAG methods in which the external knowledge is organized as a graph and retrieval exploits that structure, rather than treating the corpus as a flat set of independent chunks [18]. By retrieving connected entities and the relations between them, Graph RAG captures relational knowledge and supports more precise, context-aware responses [18]. The two paradigms are therefore complementary: vector RAG excels at locating locally relevant passages, whereas Graph RAG adds the relational and global structure that vector similarity discards [4], [18], as contrasted in Figure 1.3.

The practical benefits have been demonstrated from both ends of the design space. At the "global" end, building an entity knowledge graph and pre-computing community summaries lets the system answer corpus-wide sensemaking questions that naive RAG cannot, improving the comprehensiveness and diversity of answers [4]. At the efficiency end, incorporating graph structure into indexing and using a dual-level retrieval scheme

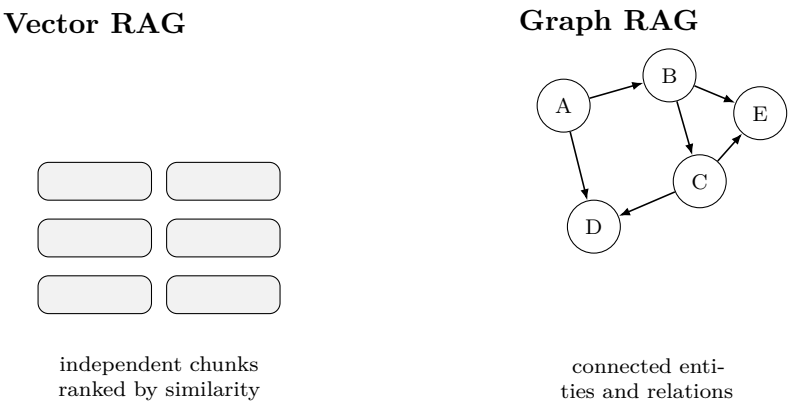

Figure 1.3: Vector RAG treats the corpus as independent chunks ranked by similarity, whereas Graph RAG retrieves connected entities and the relations between them, preserving relational structure [18].

has been shown to improve both accuracy and response time relative to conventional RAG [8]. More broadly, unifying language models with knowledge graphs is argued to combine the fluent generation of the former with the explicit, verifiable factual knowledge of the latter [17].

1.5 Report Structure

Chapter 2 covers knowledge graph construction; Chapter 3 covers retrieval and reasoning over graphs; Chapter 4 covers grounding generation; and Chapter 5 concludes with evaluation and future work.

Chapter 2

Knowledge Graph Construction

2.1 Entity and Relation Extraction

Constructing a knowledge graph from unstructured text is fundamentally a knowledge-acquisition problem: entities and the relations between them must be identified and normalized before they can be stored as triples [10]. This pipeline is typically decomposed into named entity recognition, relation extraction, and the linking steps that resolve mentions to canonical graph nodes [10]. The recent trend is to drive these steps with large language models, exploiting their broad linguistic competence to extract structured knowledge with little task-specific supervision [17].

2.1.1 Named Entity Recognition

Named entity recognition (NER) locates spans of text that mention entities and assigns them a type, and it is the foundation on which relation extraction and downstream graph population depend [15]. Contemporary approaches are dominated by deep neural architectures that learn contextual representations of tokens, superseding earlier feature-engineered models [15]. In the Graph RAG setting these extracted entities become the candidate nodes of the knowledge graph [4].

2.1.2 Relation Extraction with LLMs

Relation extraction determines how the recognized entities are connected, producing the predicate of each triple. In modern Graph RAG construction this is frequently performed by prompting a large language model to read source text and emit entity–relation–entity triples directly, an approach used to derive the entity knowledge graph in the GraphRAG pipeline [4]. Using language models as extractors is attractive because they generalize across domains and relation types without dedicated training data, a capability highlighted in roadmaps for unifying language models and knowledge graphs [17].

2.1.3 Coreference Resolution and Entity Linking

Raw extraction yields many surface mentions that refer to the same real-world entity, so two normalization steps are required to avoid a fragmented graph. Coreference resolution groups mentions—including pronouns—that denote the same entity within and across documents [13], while entity linking maps each mention to its canonical identifier in a reference knowledge base [22]. Performing these steps well is what merges duplicate nodes into a single coherent entity and gives the resulting graph its connectivity [10], [22].

2.2 Ontology Design and Schema Modeling

An ontology supplies the schema that the extracted facts must conform to, specifying the conceptualization of the domain in terms of classes, properties, and constraints [7]. Committing to such a shared vocabulary is what gives the knowledge graph consistent meaning and enables validation and inference over it [7], [9]. In a Graph RAG system the ontology therefore acts both as a target structure that guides extraction and as a contract that later retrieval and verification can rely on [9], [17].

2.2.1 Top-Down vs. Bottom-Up Schema Design

Ontology engineering can proceed top-down, where domain experts define the classes and relations in advance, or bottom-up, where the schema emerges from the entities and relations discovered in the data; in practice the two are combined, because a knowledge graph must balance a principled model with the messiness of real sources [9]. The choice affects how tightly extraction is constrained: a rich predefined schema improves consistency but can miss unanticipated relations, whereas an open, data-driven schema captures more but requires later consolidation [9], [10].

2.2.2 Reusing Existing Ontologies

Because interoperability is a central goal of the Semantic Web, established vocabularies and ontologies should be reused rather than reinvented wherever possible, allowing data from different sources to be integrated under shared semantics [2]. The RDF and OWL standards exist precisely to make such vocabularies portable and machine-interpretable across systems [24], [25], and surveys of knowledge graphs document a large ecosystem of reusable ontologies built on this foundation [9].

2.3 Graph Storage and Indexing

Once triples are extracted and normalized they must be stored in a system that supports efficient traversal and querying, and the choice of data model shapes what retrieval can do [1]. Graph database models have been surveyed extensively, and they differ in how they represent nodes, edges, and the attributes attached to them [1]. For Graph RAG the requirement is twofold: the store must both hold the relational structure and expose it to the retriever at query time [18].

2.3.1 Triple Stores vs. Property Graphs

Two dominant paradigms exist. RDF triple stores represent data strictly as (subject, predicate, object) statements aligned with the W3C standards [24], which maximizes interoperability and standardized reasoning. Property graphs instead attach arbitrary key–value properties to both nodes and edges, a model popularized for its flexibility and developer ergonomics in connected-data applications [20]. Surveys of graph data models frame these as points on a spectrum of expressiveness and standardization rather than mutually exclusive choices [1], [9].

2.3.2 Hybrid Vector-Graph Indexes

Modern Graph RAG systems index the graph alongside the vector embeddings used by dense retrieval, so that a query can exploit both semantic similarity and explicit structure [18]. LightRAG, for example, incorporates graph structures into the text index and combines them with vector representations to retrieve related entities and their relationships efficiently, using an incremental update algorithm to keep the index current as new data arrives [8]. Such hybrid indexes are what let a single system serve both the local, similarity-driven and the global, structure-driven retrieval patterns [8], [18].

Chapter 3

Retrieval and Reasoning over Graphs

3.1 Subgraph Retrieval and Traversal

In Graph RAG the retrieval target is not a list of independent passages but a relevant *subgraph* of connected entities and relations, which is then handed to the generator as structured context [18]. A common strategy is to first identify seed nodes—for instance the entities mentioned in the query, located by dense similarity—and then traverse outward along edges to gather the neighbourhood of facts that surround them [18]. LightRAG operationalizes this with a dual-level retrieval scheme that combines low-level retrieval of specific entities with high-level retrieval of broader relational context, so that both fine-grained and thematic information are captured in a single pass [8]. Retrieving a connected subgraph rather than isolated chunks is precisely what lets the model see how the retrieved facts relate to one another [18].

3.2 Community Detection and Summarization

Global, corpus-wide questions cannot be answered by retrieving a few local neighbourhoods, so Graph RAG addresses them by partitioning the entity graph into communities and summarizing each one in advance [4]. This strategy converts an intractable whole-corpus summarization task into a tractable hierarchy of partial summaries that can be composed at query time [4].

3.2.1 Hierarchical Community Clustering

The graph is divided into groups of closely related entities using community detection, and doing so hierarchically yields communities at multiple levels of granularity [4]. The underlying algorithms come from network science: the Louvain method popularized fast modularity-based community detection in very large networks [3], and the Leiden algorithm refined it to guarantee well-connected communities, which is the variant commonly used

in Graph RAG indexing [23]. The hierarchy lets the same index serve both broad and narrow global questions [4].

3.2.2 Map-Reduce over Community Summaries

At indexing time the system pre-generates a natural-language summary for every community [4]. To answer a global query it then follows a map-reduce pattern: each community summary is used to generate a partial response (the map step), and those partial responses are aggregated into a single final answer (the reduce step) [4]. For sensemaking questions over corpora in the million-token range, this approach produced substantial gains in the comprehensiveness and diversity of answers relative to a naive RAG baseline [4]. Figure 3.1 sketches the map-reduce flow over community summaries.

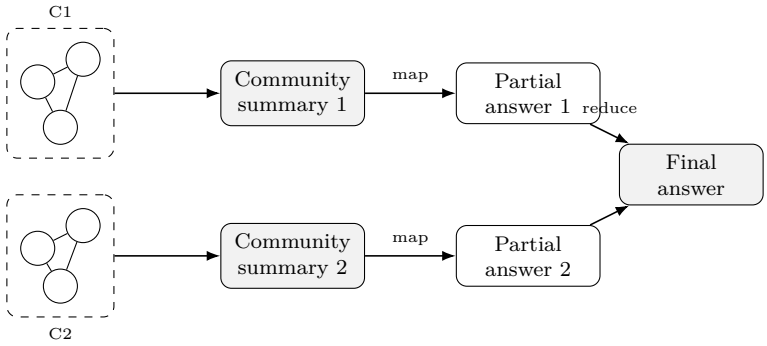

Figure 3.1: Global Graph RAG detects entity communities, pre-generates a summary per community, then maps each summary to a partial answer and reduces them into a final response [4].

3.3 Multi-Hop Reasoning and Query Planning

Many realistic questions require combining several facts that no single passage contains, a setting formalized by multi-hop question answering benchmarks that demand reasoning over multiple supporting documents [26]. A knowledge graph is naturally suited to this task because a multi-hop question maps onto a path or subgraph, and following labelled edges makes each reasoning step explicit and traceable [18]. This explicitness is a key advantage of graph-structured retrieval over flat vector search, whose similarity scores give no principled way to chain intermediate facts together [18].

Because such questions must be decomposed before they can be traversed, Graph RAG systems increasingly plan retrieval: complex queries are broken into simpler sub-queries whose results are combined, and language models are used to drive this planning over the graph [18]. Coupling the reasoning ability of language models with the structured, verifiable knowledge of a graph is exactly the complementarity emphasized in roadmaps for unifying the two paradigms [17].

3.4 Hybrid Retrieval Strategies

Graph and vector retrieval have complementary strengths, so effective systems combine them rather than choosing one: dense similarity is used to enter the graph and locate relevant entities, while graph traversal expands that entry point into a connected, relationally coherent context [18]. LightRAG embodies this by integrating graph structures with vector representations in a single index, which it reports improves both retrieval accuracy and response time over conventional RAG [8].

Hierarchical summarization can also be seen as a form of hybrid retrieval over levels of abstraction rather than over data models. RAPTOR, for example, recursively embeds, clusters, and summarizes text to build a tree, and retrieving from different levels of that tree integrates information at multiple granularities, yielding significant gains over flat retrieval [21]. Whether the structure is a knowledge graph or a summary tree, the common principle is that combining semantic similarity with explicit structure retrieves more complete and better-connected evidence than similarity alone [18], [21].

Chapter 4

Grounding Generation with Graph Context

4.1 Context Assembly and Prompt Construction

Once a relevant subgraph has been retrieved, it must be serialized into a form the language model can consume, since the generator ultimately conditions on a textual prompt [14]. In Graph RAG this means linearizing entities, relations, and any attached community summaries into the context window, so that the structure retrieved from the graph is preserved as far as possible in the prompt [18]. The design of this step matters because the answer is conditioned jointly on the query and the assembled evidence; poorly organized context degrades the generation even when the right facts were retrieved [6].

Because context windows are finite, assembly also involves selection and ordering: the GraphRAG global pipeline, for instance, first generates partial answers from individual community summaries and only then combines them, rather than attempting to fit an entire corpus into one prompt [4]. Dual-level retrieval schemes similarly assemble both specific entities and broader relational context so that the prompt reflects both local detail and global theme [8].

4.2 Ontology-Guided Verification

A distinctive advantage of grounding generation in a knowledge graph is that the graph and its ontology provide an explicit, machine-checkable reference against which generated statements can be validated [17]. Because a knowledge graph stores rich factual knowledge in a structured form, it can serve as external, verifiable knowledge for a language model, supplying both grounding and a basis for interpretability that black-box parametric models lack [17]. The ontology strengthens this further: its classes, property constraints, and logical axioms define which assertions are well-formed, so that reasoning can flag statements

that violate the schema [9], [25].

In effect, the ontology turns validation into a formal operation over the graph rather than a subjective judgement about text. This is the property that motivates unifying language models with knowledge graphs—pairing fluent generation with a substrate whose facts can be checked and whose inferences are explainable [9], [17].

4.3 Mitigating Hallucination through Structured Grounding

Hallucination—the generation of content that is fluent but unfaithful to any source—is a well-documented and pervasive failure mode of neural language generation [11]. Retrieval augmentation reduces it by conditioning generation on retrieved evidence rather than parametric memory alone [6], [14], and Graph RAG sharpens this effect by grounding the answer in explicit entities and relations whose provenance can be traced back through the graph [18]. Because a knowledge graph supplies structured, verifiable facts, unifying it with a language model directly targets the tendency of parametric models to fabricate knowledge [17].

Structured grounding also makes the residual problem measurable. Fine-grained factuality metrics decompose a generated answer into atomic facts and check each against a knowledge source [16], which aligns naturally with a graph whose triples provide exactly such atomic, checkable units [17]. Grounding therefore does more than lower the hallucination rate; it provides the reference needed to detect and quantify what remains [11], [16].

Chapter 5

Evaluation, Conclusion, and Future Work

5.1 Evaluation Metrics for Graph RAG

Evaluating a Graph RAG system requires assessing two coupled components—the retriever and the generator—because an end-to-end answer can fail at either stage [5]. Reference-free frameworks such as RAGAS were introduced precisely to evaluate RAG pipelines along these dimensions without relying on human-labelled ground truth, enabling faster iteration [5]. For global, summarization-style questions where no single gold answer exists, GraphRAG instead measures qualities such as the comprehensiveness and diversity of the generated response, typically via head-to-head LLM-based comparison [4].

5.1.1 Retrieval Quality

Retrieval quality captures whether the system surfaced the evidence actually needed to answer the query; RAGAS operationalizes this with context-oriented metrics that assess the relevance of the retrieved material [5]. This dimension is decisive because the final answer is upper-bounded by what was retrieved—missing evidence cannot be recovered downstream [6].

5.1.2 Answer Faithfulness and Groundedness

Faithfulness measures whether the generated answer is actually supported by the retrieved context rather than invented, and it is a core reference-free metric in RAGAS [5]. It can be made fine-grained by decomposing the answer into atomic facts and verifying each against the source, as in atomic factual precision scoring [16]. Since hallucination remains a pervasive risk in generation, faithfulness is the metric most directly aligned with the purpose of grounding [11].

5.2 Limitations

The benefits of Graph RAG come at the cost of building and maintaining the graph itself. Constructing a knowledge graph from text requires a multi-stage extraction pipeline—entity recognition, relation extraction, coreference resolution, and entity linking—each of which introduces errors that propagate into the graph [10], [22]. Graph-based indexing further adds up-front cost, since methods that derive an entity graph and pre-generate community summaries invoke a language model many times over the corpus before any query is served [4].

The graph must also be kept consistent and current: knowledge graphs are dynamic, and integrating new or changing data without corrupting existing structure is a recognized challenge [9]. Systems such as LightRAG explicitly introduce incremental update algorithms to address this, which is itself evidence that maintenance is a first-order concern rather than an afterthought [8]. Finally, ontology design demands domain expertise, and an inadequate schema limits the consistency and inferential power the graph can provide [7], [9].

5.3 Future Directions

Several directions follow from the limitations above and from open problems identified in the literature:

- **Cheaper, more robust construction.** Reducing the cost and error of turning text into graphs—by improving LLM-based extraction and linking—remains central to making Graph RAG practical at scale [10], [17].
- **Deeper LLM–KG integration.** Roadmaps for unifying language models and knowledge graphs point toward systems where the two are combined throughout inference rather than merely at retrieval time, so that generation and structured reasoning reinforce each other [17].
- **Efficient and incremental indexing.** Making graph indexing faster and updatable, as begun by lightweight and incremental approaches, is needed for deployment in changing data environments [8].
- **Standardized evaluation.** Consolidating reference-free metrics for both retrieval quality and answer faithfulness into shared benchmarks would let Graph RAG variants be compared reliably [5], [16].
- **Stronger factual guarantees.** Given the persistence of hallucination, using the ontology for verification and provenance is a promising route to answers that are not only grounded but checkable [11], [17].

References

[1] R. Angles and C. Gutierrez, "Survey of graph database models," *ACM Computing Surveys*, vol. 40, no. 1, pp. 1–39, 2008.

[2] T. Berners-Lee, J. Hendler, and O. Lassila, "The semantic web," *Scientific American*, vol. 284, no. 5, pp. 34–43, 2001.

[3] V. D. Blondel, J.-L. Guillaume, R. Lambiotte, and E. Lefebvre, "Fast unfolding of communities in large networks," *Journal of Statistical Mechanics: Theory and Experiment*, vol. 2008, no. 10, P10008, 2008.

[4] D. Edge, H. Trinh, N. Cheng, *et al.*, "From local to global: A graph RAG approach to query-focused summarization," *arXiv preprint arXiv:2404.16130*, 2024.

[5] S. Es, J. James, L. Espinosa-Anke, and S. Schockaert, "RAGAS: Automated evaluation of retrieval augmented generation," in *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics (EACL): System Demonstrations*, 2024.

[6] Y. Gao, Y. Xiong, X. Gao, *et al.*, "Retrieval-augmented generation for large language models: A survey," *arXiv preprint arXiv:2312.10997*, 2023.

[7] T. R. Gruber, "A translation approach to portable ontology specifications," *Knowledge Acquisition*, vol. 5, no. 2, pp. 199–220, 1993.

[8] Z. Guo, L. Xia, Y. Yu, T. Ao, and C. Huang, "LightRAG: Simple and fast retrieval-augmented generation," *arXiv preprint arXiv:2410.05779*, 2024.

[9] A. Hogan, E. Blomqvist, M. Cochez, *et al.*, "Knowledge graphs," *ACM Computing Surveys*, vol. 54, no. 4, pp. 1–37, 2021.

[10] S. Ji, S. Pan, E. Cambria, P. Marttinen, and P. S. Yu, "A survey on knowledge graphs: Representation, acquisition, and applications," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 33, no. 2, pp. 494–514, 2022.

[11] Z. Ji, N. Lee, R. Frieske, *et al.*, "Survey of hallucination in natural language generation," *ACM Computing Surveys*, vol. 55, no. 12, pp. 1–38, 2023.

[12] V. Karpukhin, B. Oğuz, S. Min, *et al.*, "Dense passage retrieval for open-domain question answering," in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2020.

[13] K. Lee, L. He, M. Lewis, and L. Zettlemoyer, "End-to-end neural coreference resolution," in *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2017.

[14] P. Lewis, E. Perez, A. Piktus, *et al.*, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[15] J. Li, A. Sun, J. Han, and C. Li, "A survey on deep learning for named entity recognition," *IEEE Transactions on Knowledge and Data Engineering*, vol. 34, no. 1, pp. 50–70, 2022.

[16] S. Min, K. Krishna, X. Lyu, *et al.*, "FActScore: Fine-grained atomic evaluation of factual precision in long form text generation," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023.

[17] S. Pan, L. Luo, Y. Wang, C. Chen, J. Wang, and X. Wu, "Unifying large language models and knowledge graphs: A roadmap," *IEEE Transactions on Knowledge and Data Engineering*, vol. 36, no. 7, pp. 3580–3599, 2024.

[18] B. Peng, Y. Zhu, Y. Liu, *et al.*, "Graph retrieval-augmented generation: A survey," *arXiv preprint arXiv:2408.08921*, 2024.

[19] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using siamese BERT-networks," in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2019.

[20] I. Robinson, J. Webber, and E. Eifrem, *Graph Databases: New Opportunities for Connected Data*, 2nd ed. O'Reilly Media, 2015.

[21] P. Sarthi, S. Abdullah, A. Tuli, S. Khanna, A. Goldie, and C. D. Manning, "RAPTOR: Recursive abstractive processing for tree-organized retrieval," in *International Conference on Learning Representations (ICLR)*, 2024.

[22] W. Shen, J. Wang, and J. Han, "Entity linking with a knowledge base: Issues, techniques, and solutions," *IEEE Transactions on Knowledge and Data Engineering*, vol. 27, no. 2, pp. 443–460, 2015.

[23] V. A. Traag, L. Waltman, and N. J. van Eck, "From louvain to leiden: Guaranteeing well-connected communities," *Scientific Reports*, vol. 9, p. 5233, 2019.

[24] W3C, "RDF 1.1 concepts and abstract syntax," World Wide Web Consortium (W3C), W3C Recommendation, 2014.

[25] W3C OWL Working Group, "OWL 2 web ontology language document overview," World Wide Web Consortium (W3C), W3C Recommendation, 2012.

[26] Z. Yang, P. Qi, S. Zhang, *et al.*, "HotpotQA: A dataset for diverse, explainable multi-hop question answering," in *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2018.

Appendix A

Supplementary Material

Place additional derivations, code, or data here. Appendix chapters are lettered automatically because of `\appendix` in `main.tex`.